

Dual Core RV32I AXI4 Lite SoC

Project Documentation and Evaluation Report

Project 04

Group Members

Muhammad Saad

Faraz Ismail

Alishba Khan

September 2026

Project Summary

This project implements a small dual-core RISC-V system-on-chip using two synthesizable RV32I cores and an AXI4-Lite interconnect. Each core has a private instruction memory and both cores share data memory, a mailbox, a UART, and an invalid-address response slave. Separate read and write arbiters resolve simultaneous requests. The design therefore demonstrates the required multi-core routing, arbitration, shared-memory access, and inter-core communication without attempting a full packet-switched Network-on-Chip.

The RTL was checked with directed self-checking testbenches and four UVM environments covering the RV32I core, AXI4-Lite master, UART peripheral, and integrated dual-core SoC. The RV32I environment has a recorded passing run with coverage and scoreboard results. The full-SoC environment programs both cores and checks shared SRAM, mailbox communication, contention, UART routing, reset, and invalid-address handling. Its three program payloads were validated against the RTL, while a final Questa transcript and coverage database have not yet been recorded. FPGA build files were also prepared for Arty A7-100T and Zybo. A complete SKY130A physical-design flow was run in LibreLane; physical checks and hold timing pass, while three slow-process corners retain setup violations.

Evaluation Summary

Area	Result	Evaluation
Integrated RTL	Two cores, AXI4-Lite fabric, SRAM, mailbox, UART, error slave	Implemented
Directed verification	Standalone blocks, contention, peripherals, errors, and dual-core operation	Passed
UVM verification	RV32I core, AXI master, UART, and full-SoC environments	Core pass recorded; remaining environments included
Physical design	Complete 77-stage SKY130A LibreLane run	Completed
Physical sign-off checks	DRC, LVS, antenna, connectivity, and hold	Passed
Setup timing	Passes typical and fast corners; worst slow-corner WNS -5.019 ns	Partial
FPGA implementation	Arty A7-100T and Zybo wrappers, constraints, programs, and build files	Prepared; hardware validation pending

Project Objectives

- Integrate two independently operating 32-bit RV32I processors.
- Connect both processors to shared resources through AXI4-Lite.
- Route requests by address and arbitrate simultaneous read and write traffic.
- Demonstrate communication through shared SRAM and mailbox registers.
- Provide UART status and defined error responses for invalid accesses.
- Verify normal operation, contention, reset, and error cases.
- Complete synthesis and physical design and report the measured results.

System Architecture

Both RV32I cores fetch instructions from private memories. Their data-side requests enter separate AXI4-Lite adapters and then a common fabric. The fabric decodes the target address, arbitrates requests from the two masters, forwards each transaction to one slave, and returns the matching response to the requesting core.

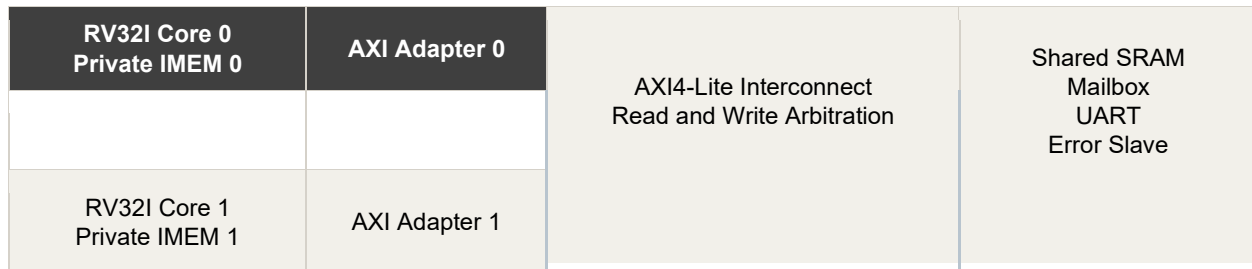


Figure 1 Implemented dual-core data path and shared-resource structure

Address Map

Address range	Destination	Purpose
0x1000_0000 - 0x1000_FFFF	Shared SRAM	Shared program data and core-to-core data exchange
0x2000_0000 - 0x2000_00FF	Mailbox	Bidirectional messages, pending flags, and IRQ control
0x3000_0000 - 0x3000_00FF	UART	Transmit, receive, status, and control registers
0x3000_1000 - 0x3000_10FF	Reserved SPI	Returns AXI DECERR in the current design
All other addresses	Error slave	Returns AXI DECERR

Interconnect Operation

Read and write transactions use independent round-robin arbiters. Once a master wins a channel, ownership remains fixed until the response completes. This avoids response mixing when both cores are active. The AXI adapter also keeps write-address and write-data handshakes independent and prevents a held core request from being sent twice.

RTL Implementation

RV32I Cores

The cores support the RV32I instruction groups used by the test programs: register and immediate ALU operations, conditional branches, LUI, AUIPC, JAL, JALR, loads, and stores. Loads and stores hold the program counter until the external memory response arrives. Byte and halfword accesses use aligned word data with the correct byte strobes and sign extension. EBREAK marks successful program completion. A misaligned access or bus error asserts both fail and done because a trap handler is outside the project scope.

Memory and Peripherals

The functional RTL keeps a 1,024-word instruction memory in each core and a 64 KiB shared data SRAM. The mailbox contains two message directions, pending bits, write-one-to-clear status, and interrupt enables. The UART provides transmit, receive, status, and control registers and uses an 8E1 serial frame. Invalid and reserved accesses terminate cleanly through the error slave instead of leaving the bus waiting.

Reusable Modules and New Integration

The ALU, branch comparator, control unit, immediate generator, program counter, register file, baud-rate generator, transmitter, and receiver were reused from earlier work. The core wrapper, AXI master, and SRAM were adjusted for external shared-bus operation. The address decoder, arbiters, interconnect, SoC fabric, mailbox, UART slave, SRAM slave, error slave, and top level provide the project-specific integration.

Functional and ASIC Memory Configurations

Target	Private instruction memory	Shared data memory	Reason
Functional RTL	2 x 1,024 x 32 bits	16,384 x 32 bits (64 KiB)	Full simulation-oriented capacity
First ASIC attempt	2 x 64 x 32 bits	256 x 32 bits	Initial reduction for standard-cell implementation
Final ASIC run	2 x 32 x 32 bits	64 x 32 bits	Fits supplied programs and reduces mux and clock-network cost

The final ASIC memories are synthesized from standard cells rather than compiled SRAM macros. This reduced configuration is an implementation tradeoff for the available PDK, tools, and runtime. The address map and communication behavior remain unchanged.

Directed Verification

The directed environment uses plain, standalone Verilog testbenches. Each testbench creates its own stimulus, timeout, checks, and PASS or FAIL result. This keeps the verification files easy to follow while still covering reset, normal traffic, simultaneous requests, error responses, and peripheral behavior.

Verification area	Main checks
Core execution	RV32I arithmetic, control flow, byte and halfword loads and stores, completion and failure
AXI master	Read and write channel ordering, delayed response, and duplicate-request prevention
Decoder and arbiters	Boundary addresses, one-hot routing, fairness, simultaneous requests, and response ownership
Shared SRAM	Reads, full and partial writes, byte strobes, invalid cases, and reset behavior
Mailbox	Both message directions, pending status, interrupt enable, clear, and reset
UART	Transmit and receive loopback, busy/status, receive valid, parity error, and frame error
Integrated SoC	Both cores running together, shared-memory contention, mailbox exchange, and clean completion

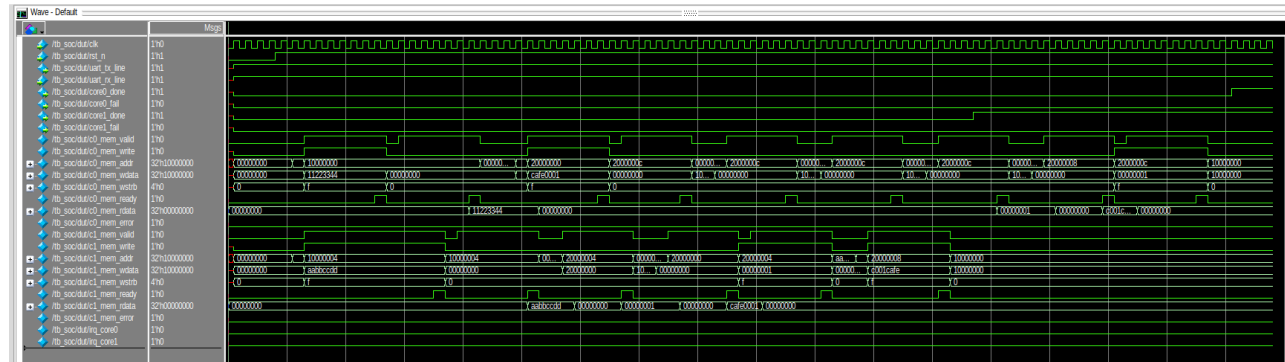


Figure 2 Final dual-core SoC waveform. Both cores complete with done asserted and fail low.

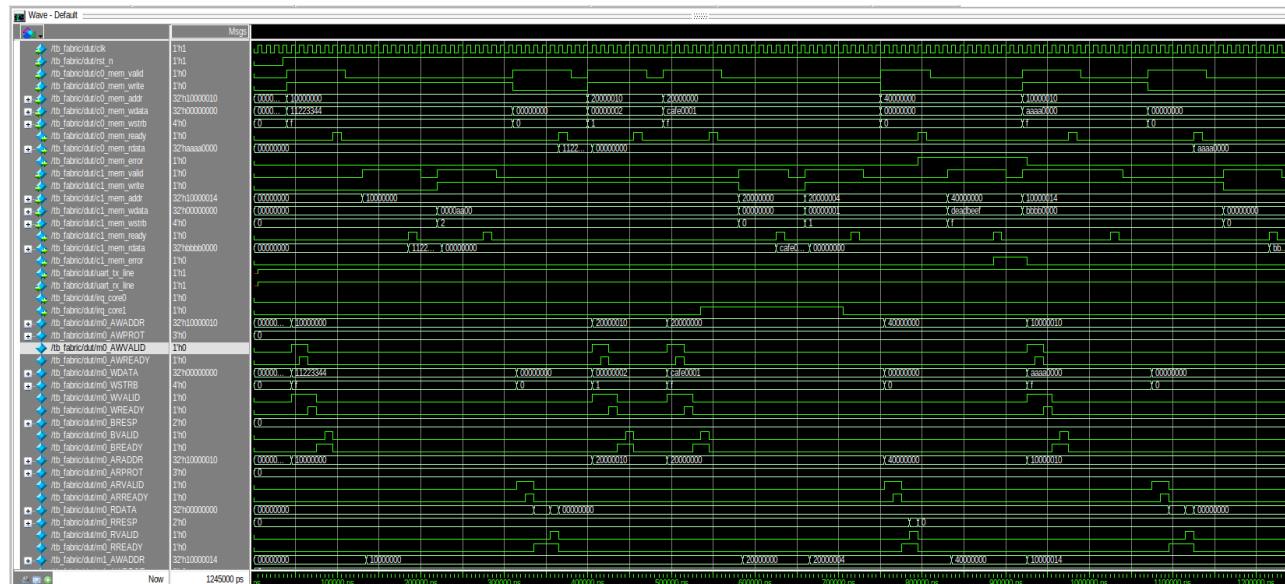


Figure 3 Fabric-level waveform showing two master interfaces and routed shared-resource activity.

Peripheral Verification

The mailbox waveform shows the two message directions, pending and interrupt behavior, status clearing, and reset. The UART waveform shows memory-mapped register activity together with serial transmit and receive behavior. These captures provide direct evidence for the two external communication mechanisms required by the project.

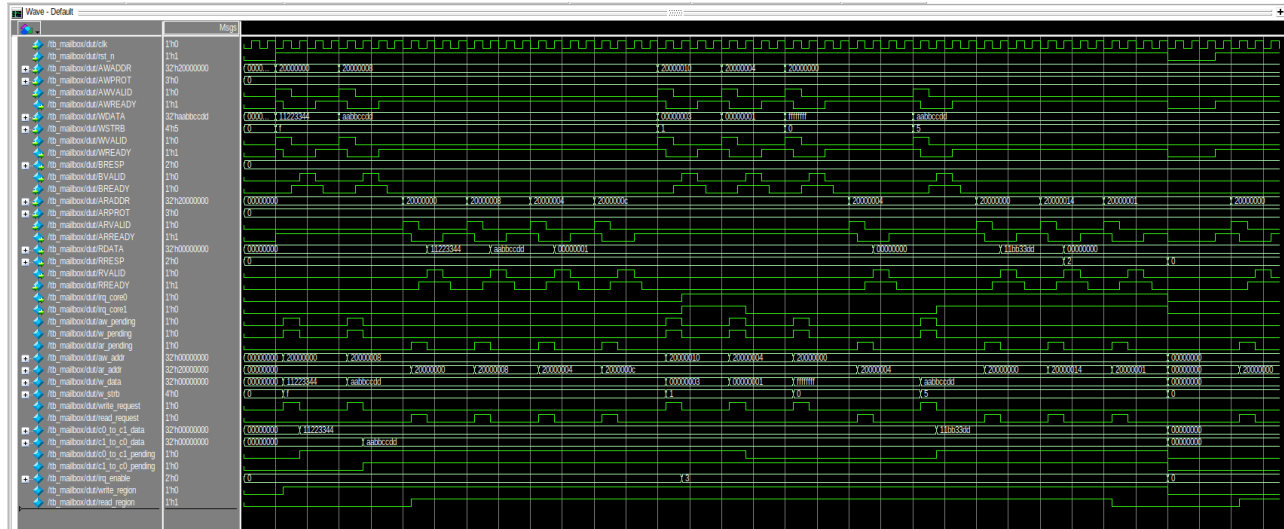


Figure 4 Mailbox verification with bidirectional data, pending status, interrupt control, clear, and reset cases.

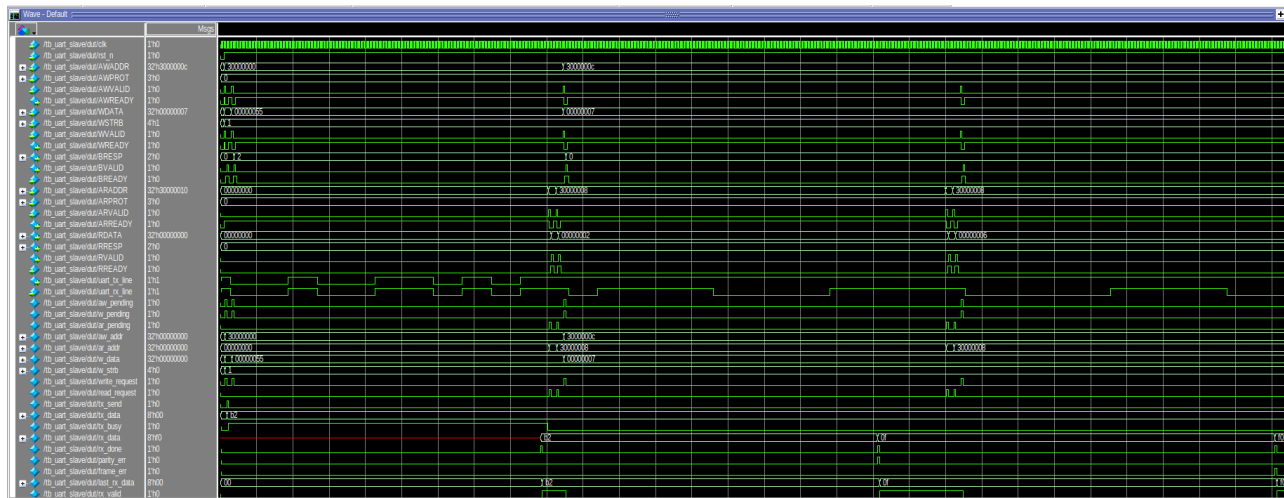


Figure 5 UART verification showing register access and transmit-receive loopback activity.

UVM Core Verification

A separate UVM environment verifies the standalone RV32I core. It contains an interface, sequence item, sequences, driver, monitor, agent, scoreboard and reference model, coverage collector, assertions, environment, and test. The driver loads instructions and models a 4 KiB data memory with variable response delay and injected error cases.

The run combines 31 directed programs with 10 constrained-random programs. Two programs are intentionally interrupted by reset, so 39 complete normally and two are recorded as reset aborts. This is expected behavior, not a failure.

UVM metric	Final result
Planned programs	41
Completed programs	39
Intentional reset aborts	2
Instructions checked	3,700
Memory transfers	497
Scoreboard checks	2,131,669
Mismatches	0
Instruction, outcome, and memory coverage	100.00% each
Warnings, errors, and fatals	0, 0, and 0
Final result	RV32I_UVM_RESULT: PASS

```
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(+struct)
# UVM_INFO @ 0: reporter [RV32I] Running test rv32i_test...
# UVM_INFO driver.sv(95) @ 0: uvm_test_top.env.agent.drv [PROGRAM] Loading smoke (30 instructions)
# UVM_INFO driver.sv(95) @ 11270000: uvm_test_top.env.agent.drv [PROGRAM] Loading alu (445 instructions)
# UVM_INFO driver.sv(95) @ 30480000: uvm_test_top.env.agent.drv [PROGRAM] Loading branch (184 instructions)
# UVM_INFO driver.sv(95) @ 44310000: uvm_test_top.env.agent.drv [PROGRAM] Loading memory (45 instructions)
# UVM_INFO driver.sv(95) @ 57010000: uvm_test_top.env.agent.drv [PROGRAM] Loading memory_min_latency (45 instructions)
# UVM_INFO driver.sv(95) @ 68860000: uvm_test_top.env.agent.drv [PROGRAM] Loading memory_long_latency (45 instructions)
# UVM_INFO driver.sv(95) @ 88740000: uvm_test_top.env.agent.drv [PROGRAM] Loading lh_odd (5 instructions)
# UVM_INFO driver.sv(95) @ 99110000: uvm_test_top.env.agent.drv [PROGRAM] Loading lhu_odd (5 instructions)
# UVM_INFO driver.sv(95) @ 109480000: uvm_test_top.env.agent.drv [PROGRAM] Loading lw_1 (5 instructions)
# UVM_INFO driver.sv(95) @ 119850000: uvm_test_top.env.agent.drv [PROGRAM] Loading lw_2 (5 instructions)
# UVM_INFO driver.sv(95) @ 130220000: uvm_test_top.env.agent.drv [PROGRAM] Loading lw_3 (5 instructions)
# UVM_INFO driver.sv(95) @ 140590000: uvm_test_top.env.agent.drv [PROGRAM] Loading sh_odd (5 instructions)
# UVM_INFO driver.sv(95) @ 150960000: uvm_test_top.env.agent.drv [PROGRAM] Loading sw_1 (5 instructions)
# UVM_INFO driver.sv(95) @ 161330000: uvm_test_top.env.agent.drv [PROGRAM] Loading sw_2 (5 instructions)
# UVM_INFO driver.sv(95) @ 171700000: uvm_test_top.env.agent.drv [PROGRAM] Loading sw_3 (5 instructions)
# UVM_INFO driver.sv(95) @ 182070000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_load (5 instructions)
# UVM_INFO driver.sv(95) @ 192440000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_load_6 (5 instructions)
# UVM_INFO driver.sv(95) @ 202810000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_load_7 (5 instructions)
# UVM_INFO driver.sv(95) @ 213180000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store (5 instructions)
# UVM_INFO driver.sv(95) @ 223550000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_4 (5 instructions)
# UVM_INFO driver.sv(95) @ 233920000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_5 (5 instructions)
# UVM_INFO driver.sv(95) @ 244290000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_6 (5 instructions)
# UVM_INFO driver.sv(95) @ 254660000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_7 (5 instructions)
# UVM_INFO driver.sv(95) @ 265030000: uvm_test_top.env.agent.drv [PROGRAM] Loading read_error (5 instructions)
# UVM_INFO driver.sv(95) @ 275410000: uvm_test_top.env.agent.drv [PROGRAM] Loading write_error (5 instructions)
# UVM_INFO driver.sv(95) @ 285800000: uvm_test_top.env.agent.drv [PROGRAM] Loading late_read_error (6 instructions)
# UVM_INFO driver.sv(95) @ 296170000: uvm_test_top.env.agent.drv [PROGRAM] Loading out_of_range (3 instructions)
# UVM_INFO driver.sv(95) @ 306540000: uvm_test_top.env.agent.drv [PROGRAM] Loading reset_read (3 instructions)
# UVM_INFO driver.sv(95) @ 317040000: uvm_test_top.env.agent.drv [PROGRAM] Loading reset_write (4 instructions)
# UVM_INFO driver.sv(95) @ 327410000: uvm_test_top.env.agent.drv [PROGRAM] Loading reset_recovery (3 instructions)
# UVM_INFO driver.sv(95) @ 337840000: uvm_test_top.env.agent.drv [PROGRAM] Loading implementation_nops (6 instructions)
# UVM_INFO driver.sv(95) @ 348270000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 358700000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 369130000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 379560000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 390000000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 400430000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 410860000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 421290000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 431720000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 442150000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 452580000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 463010000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 473440000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 483870000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 494300000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 504730000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 531920000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO coverage.sv(39) @ 531920000: uvm_test_top.env.cov [COVERAGE] Instruction group=100.00% outcome group=100.00%
# UVM_INFO coverage.sv(63) @ 531920000: uvm_test_top.env.mem_cov [MEM_COVERAGE] Memory group=100.00%
# UVM_INFO scoreboard.sv(259) @ 531920000: uvm_test_top.env.scb [SCB_SUMMARY] plans=41 completed=39 aborted=2 instructions=3700 transfers=497 checks=2131669 mismatches=0
```

Figure 6 UVM transcript showing the completed test list, coverage totals, and scoreboard statistics.

UVM Result Evidence

```

# UVM_INFO driver.sv(95) @ 233920000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_4 (5 instructions)
# UVM_INFO driver.sv(95) @ 244290000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_5 (5 instructions)
# UVM_INFO driver.sv(95) @ 254660000: uvm_test_top.env.agent.drv [PROGRAM] Loading bad_store_7 (5 instructions)
# UVM_INFO driver.sv(95) @ 265030000: uvm_test_top.env.agent.drv [PROGRAM] Loading read_error (5 instructions)
# UVM_INFO driver.sv(95) @ 275410000: uvm_test_top.env.agent.drv [PROGRAM] Loading write_error (5 instructions)
# UVM_INFO driver.sv(95) @ 285800000: uvm_test_top.env.agent.drv [PROGRAM] Loading late_read_error (6 instructions)
# UVM_INFO driver.sv(95) @ 296300000: uvm_test_top.env.agent.drv [PROGRAM] Loading out_of_range (3 instructions)
# UVM_INFO driver.sv(95) @ 306900000: uvm_test_top.env.agent.drv [PROGRAM] Loading reset_read (3 instructions)
# UVM_INFO driver.sv(95) @ 317040000: uvm_test_top.env.agent.drv [PROGRAM] Loading reset_write (4 instructions)
# UVM_INFO driver.sv(95) @ 327410000: uvm_test_top.env.agent.drv [PROGRAM] Loading reset_recovery (3 instructions)
# UVM_INFO driver.sv(95) @ 337840000: uvm_test_top.env.agent.drv [PROGRAM] Loading implementation_nops (6 instructions)
# UVM_INFO driver.sv(95) @ 348270000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 366030000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 384920000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 403380000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 421880000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 440260000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 458450000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 476830000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 494920000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO driver.sv(95) @ 513030000: uvm_test_top.env.agent.drv [PROGRAM] Loading constrained_random (314 instructions)
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 531920000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO coverage.sv(39) @ 531920000: uvm_test_top.env.cov [COVERAGE] Instruction group=100.00% outcome group=100.00%
# UVM_INFO coverage.sv(63) @ 531920000: uvm_test_top.env.mem_cov [MEM_COVERAGE] Memory group=100.00%
# UVM_INFO scoreboard.sv(259) @ 531920000: uvm_test_top.env.scb [SCB_SUMMARY] plans=41 completed=39 aborted=2 instructions=3700 transfers=497 checks=2131669 mismatches=0
#
--- UVM Report Summary ---
#
Quit count : 0 of 20
# ** Report counts by severity
# UVM_INFO : 48
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [COVERAGE] 1
# [MEM_COVERAGE] 1
# [PROGRAM] 41
# [Questa UVM] 2
# [RNTST] 1
# [SCB_SUMMARY] 1
# [TEST_DONE] 1
# RV32I UVM RESULTS: PASS
# ** Note: $finish : tb_top.sv(39)
# Time: 531920 ns Iteration: 61 Instance: /tb_top
# Break in Module tb_top at tb_top.sv line 39
VSIM >]

```

Figure 7 Final UVM summary with PASS and zero warnings, errors, or fatal messages.

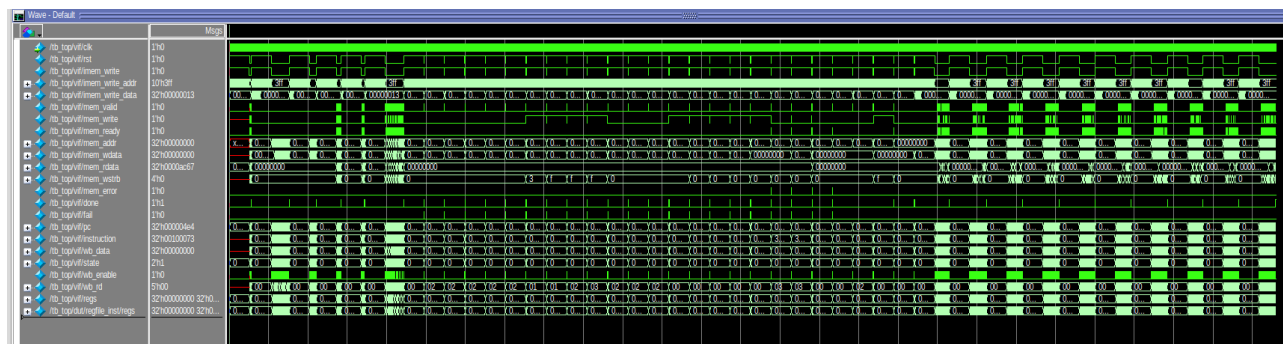


Figure 8 UVM execution waveform showing instruction loading, memory handshake, PC, state, and register activity.

The core assertions check stable requests during stalls, legal response timing, alignment and strobes, PC stability, writeback suppression on errors, sticky completion, a quiet halted core, and the fixed-zero behavior of register x0. The complete dual-core routing behavior is still verified by the directed SoC environment.

Additional UVM Environments

The AXI, UART, and full-SoC environments complement the detailed core test. The module-level environments isolate individual interfaces, while the integrated environment runs both processors through the complete interconnect and shared peripherals.

Environment	Design under test	Main verification content	Recorded evidence
RV32I core	Standalone RV32I core	31 directed and 10 constrained-random programs, memory latency and errors, reset, assertions, scoreboard, and coverage	PASS; 2,131,669 checks, zero mismatches, and 100% defined functional coverage
AXI4-Lite master	Core-to-AXI master adapter	Smoke, random, and back-to-back sequences; read/write paths, byte strobes, error responses, channel stability, scoreboard, and coverage	Source environment and saved WLF included; no final transcript or coverage export
UART	AXI4-Lite UART slave and serial path	Register access, TX busy handling, RX parity and frame errors, invalid address, clear, loopback, random, corner, reset, and stress test classes	Source environment, VCD, and saved WLF included; no final transcript or coverage export
Full dual-core SoC	Two RV32I cores and complete AXI4-Lite SoC	Concurrent SRAM and mailbox exchange, contention, UART routing, reset, unmapped access, assertions, scoreboard, and functional coverage	Source complete; all three program payloads pass RTL validation; final UVM transcript pending

Full SoC UVM Verification

The environment uses one active SoC agent. Its driver resets the design and loads independent programs into both private instruction memories. The monitor observes both core request paths and classifies SRAM, mailbox, UART, and invalid-address traffic. The scoreboard checks core completion, expected failures, shared-memory values, mailbox data and clearing, register results, and simultaneous fabric requests. Procedural assertions check response timing, request stability during stalls, and sticky completion.

Three sequences are included: a concurrent shared-SRAM and bidirectional-mailbox exchange, a UART read/write case, and an unmapped load that must return an error to core 0. Reset is applied before every case. The source is separated into item, sequence, driver, monitor, agent, scoreboard, coverage, environment, and test files, with one package file controlling their compile order.

Only the RV32I run currently has sufficient transcript and coverage evidence for a quantitative UVM pass claim. The AXI and UART folders are reported as implemented module-level environments. The full-SoC source is complete and its machine-code scenarios pass against the RTL, but its final Questa UVM run remains to be captured.

FPGA Implementation Preparation

The FPGA directory reuses the SoC RTL with the full 1,024-word instruction memory for each core and the 16,384-word shared SRAM. Separate core0.hex and core1.hex files initialize the two programs. The board wrapper divides the input clock by four and calculates the UART divider for approximately 115200 baud.

Item	Arty A7-100T	Zybo
Top module	fpga_top	fpga_top_zybo
Input and SoC clock	100 MHz input; 25 MHz SoC clock	125 MHz input; 31.25 MHz SoC clock

Item	Arty A7-100T	Zybo
Constraints	arty.xdc	zybo.xdc
Build path	F4PGA flow.json target for XC7A100TCSG324-1	Makefile legacy common.mk path
Current status	Files prepared; bitstream and board test not recorded	Files prepared; bitstream and board test not recorded

LED 0 indicates successful completion of core 0, LED 1 indicates successful completion of core 1, LED 2 latches mailbox activity, and LED 3 latches contention or reports a core failure. UART TX and RX provide the external serial connection. These files define a practical board demonstration, but no utilization, timing, bitstream, or on-board screenshot is available for the report.

ASIC Physical Design

The final ASIC flow uses LibreLane 3.0.11 with the SKY130A PDK and the sky130_fd_sc_hd standard-cell library. The initial physical-design attempt showed that the standard-cell memories created too many sequential cells, clock sinks, and wide read muxes. Post-CTS setup repair continued for hours without reaching closure. The final run reduced the memory arrays and relaxed selected implementation constraints so the complete flow could finish and provide usable sign-off evidence.

Final Implementation Configuration

Parameter	Value	Parameter	Value
Clock period	15 ns	Nominal frequency	66.67 MHz
Die area	1,100 x 1,100 um	Die size	1.21 mm2
Core bounds	30,30 to 1070,1070	Target density	50%
Clock uncertainty	0.25 ns	Maximum fanout	32
PDN horizontal	Metal 5	PDN vertical	Metal 4
CTS clustering	16 sinks	Cluster diameter	50 um

Optimization Progress

Metric	Earlier run	Final run	Change
Mapped cells	47,793	27,616	-42.22%
Mapped area	699,848.7 um2	377,366.9 um2	-46.08%
Sequential cells	15,209	7,017	-53.86%
Global placement	40 min 46.988 s	19 min 26.945 s	-52.3%
Clock tree synthesis	2 min 46.444 s	1 min 46.699 s	-35.9%
Synthesis	2 min 46.004 s	1 min 6.522 s	-60%
Flow outcome	Stalled in setup repair	77 stages completed	2 h 27 min 50.514 s

The reduction removed the shared-memory mux as the main timing bottleneck. The final critical path moved into the core instruction, decode, ALU, and register-file writeback logic. This confirmed that additional repair iterations alone would not solve the remaining slow-corner delay.

Final Routed Layout

The final 1.1 mm by 1.1 mm layout has an even standard-cell distribution and a regular power-delivery network. Routing completed with zero overflow. Detailed-route cleanup reduced the reported DRC count from 132 to 16, then 6, and finally zero.

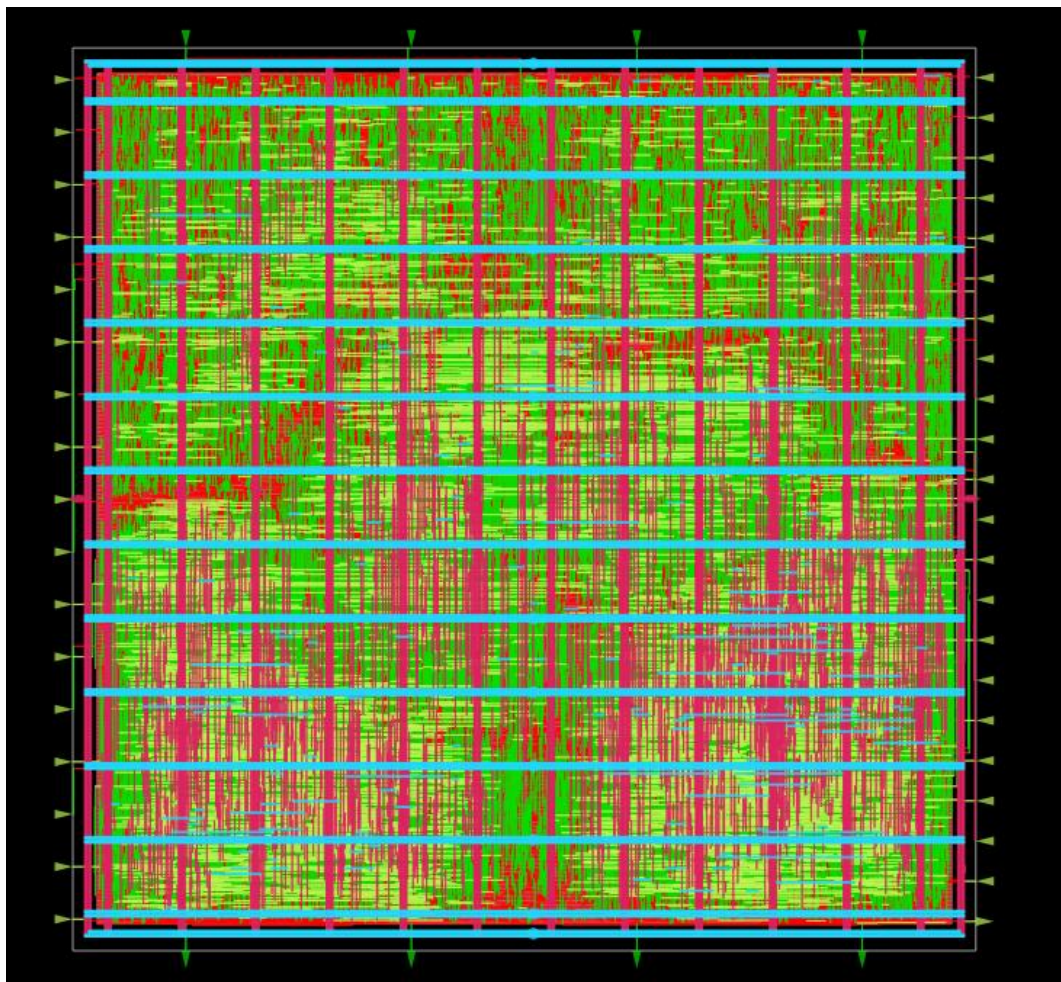


Figure 9 Final routed SKY130A layout with standard-cell placement, signal routing, and power-grid structure.

Layout metric	Final value
Final standard-cell area	Approximately 529,762 μm^2
Core utilization	49.1723%
Non-filler standard-cell instances	58,894
Global-route utilization and overflow	43.23% and 0
Detailed-route wire and vias	2,384,160 μm and 353,404 vias

Physical Design Evaluation

Sign Off and Manufacturability Checks

Check	Result	Check	Result
Magic DRC	0 violations	KLayout DRC	0 violations
LVS	Unique match	Antenna	0 net or pin violations
KLayout XOR	0 differences	Power grid	0 violations
Disconnected critical pins	0	Hold timing	All corners pass
Lint errors	0	Lint warnings	22 nonfatal warnings

Multi Corner Timing

Corner	Setup WNS ns	Setup TNS ns	Violating endpoints	Hold WNS ns
nom_tt	+4.808	0	0	+0.450
nom_ss	-4.135	-6,937.566	2,262	+0.414
nom_ff	+8.432	0	0	+0.246
min_tt	+5.267	0	0	+0.450
min_ss	-3.302	-5,058.418	2,259	+0.550
min_ff	+8.737	0	0	+0.248
max_tt	+4.316	0	0	+0.453
max_ss	-5.019	-8,836.206	2,262	+0.234
max_ff	+8.096	0	0	+0.243

The 15 ns clock meets setup timing in the typical and fast corners, and all nine corners have positive hold slack. The three slow corners fail setup, with worst WNS of -5.019 ns at max_ss. Based on the observed worst path, a conservative timing-only fallback would use at least a 21 ns clock, about 47.6 MHz. A better speed-focused revision would insert a registered writeback boundary and then optimize against the maximum-parasitic slow corner.

Power and IR Drop

Measurement	Reported value	Interpretation
Internal power	41.4452 mW	Tool estimate
Switching power	28.4617 mW	Tool estimate
Leakage power	1.1465 uW	Tool estimate
Aggregate power	69.9081 mW	Maximum-style consolidated metric
Nominal typical power	58.845 mW	Typical-corner report
Average IR drop	45.2 uV	Preliminary grid result
Worst VPWR and VGND drop	259 uV and 278.627 uV	Preliminary grid result

The power values use the implementation tool's assumed activity and are not measurements from fabricated silicon or an application-calibrated simulation. The power grid is connected and its reported drop is small, but the run did not include an explicit voltage-source location file. The IR result should therefore be treated as a preliminary grid check rather than package-aware sign-off.

Project Limitations

- The final ASIC layout does not close setup timing in the three slow-process corners at 15 ns.
- Quantitative PASS and coverage evidence is available for the RV32I core UVM run. AXI, UART, and full-SoC UVM sources are included, but their final transcripts and coverage exports are not present.
- The ASIC memories are reduced standard-cell arrays rather than the full functional memories or compiled SRAM macros.
- The processor reports alignment and bus errors through fail and done instead of a RISC-V trap handler.
- Arty A7-100T and Zybo FPGA build files are included, but no generated bitstream, implementation report, or board-level capture is included.

Conclusion

The project meets the main RTL integration goal. Two RV32I cores execute independently and share memory and peripherals through a structured AXI4-Lite interconnect. Directed simulation demonstrates routing, arbitration, contention, shared SRAM, mailbox communication, UART access, reset, and invalid-address handling. Four UVM environments are included for the RV32I core, AXI master, UART, and complete dual-core SoC. The core environment provides the recorded quantitative PASS result. The full-SoC environment adds integrated sequences, monitoring, scoreboarding, assertions, and functional coverage, with its program payloads validated directly against the RTL.

The final ASIC run produces a complete routed SKY130A implementation with clean DRC, LVS, antenna, connectivity, congestion, and hold results. Reducing the standard-cell memories cut the cell count and runtime enough to complete the full flow. Slow-corner setup timing remains the main item for further work. A registered writeback stage or a clock period near 21 ns would be the next practical paths to closure.